



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

Generic Classes and Methods

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Lecture Objectives

- ❖ Understand the problem without Generics
- ❖ Explain why type safety matters
- ❖ Recognize the need for Generic classes and methods
- ❖ Distinguish between:
- ❖ Compile-time vs Runtime errors

Introduction

- ★ Generic methods and generic classes (and interfaces) enable you to specify, with a single method declaration, a set of related methods, or with a single class declaration, a set of related types, respectively.
- ★ Generics also provide compile-time type safety that allows you to catch invalid types at compile time.

Motivation for Generic Methods

In traditional Java programming, when a method needs to operate on multiple data types, developers typically use:

- ▶ Method Overloading: define multiple methods with the same logic but different parameter types

```
// OverloadedMethods.java  
// Printing array elements using overloaded methods.
```

```
public class OverloadedMethods {  
    public static void main(String[] args) {  
        // create arrays of Integer, Double and Character  
        Integer[] integerArray = { 1, 2, 3, 4, 5, 6 };  
        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };  
        Character[] characterArray = { 'H', 'E', 'L', 'L', 'O' };  
  
        System.out.println("Array integerArray contains:");  
        printArray(integerArray); // pass an Integer array  
        System.out.println("\nArray doubleArray contains:");  
        printArray(doubleArray); // pass a Double array  
        System.out.println("\nArray characterArray contains:");  
        printArray(characterArray); // pass a Character array  
    }  
}
```

```
// method printArray to print Integer array
public static void printArray(Integer[] inputArray) {
    // display array elements
    for (Integer element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
}
```

```
// method printArray to print Double array
public static void printArray(Double[] inputArray) {
    // display array elements
    for (Double element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
}
```

```
// method printArray to print Character array
public static void printArray(Character[] inputArray) {
    // display array elements
    for (Character element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
}
```

Array integerArray contains:
1 2 3 4 5 6

Array doubleArray contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7

Array characterArray contains:
H E L L O

```
// OverloadedMethods.java  
// Printing array elements using overloaded methods.
```

```
public class OverloadedMethods {  
    public static void main(String[] args) {  
        // create arrays of Integer, Double and Character  
        Integer[] integerArray = { 1, 2, 3, 4, 5, 6 };  
        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };  
        Character[] characterArray = { 'H', 'E', 'L', 'L', 'O' };  
  
        System.out.println("Array integerArray contains:");  
        printArray(integerArray); // pass an Integer array  
        System.out.println("\nArray doubleArray contains:");  
        printArray(doubleArray); // pass a Double array  
        System.out.println("\nArray characterArray contains:");  
        printArray(characterArray); // pass a Character array  
    }  
}
```

Generic Methods: Implementation and Compile-Time Translation

- ★ If the operations performed by several overloaded methods are identical for each argument type, the overloaded methods can be more compactly and conveniently coded using a generic- method.
- ★ You can write a single generic method declaration that can be called with arguments of different types.
- ★ Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately.

Generic Methods: Implementation and Compile-Time Translation

- All generic method declarations have a type-parameter section delimited by angle brackets (< and >) that precedes the method's return type (< T > in this example).
- Each type-parameter section contains one or more type parameters (also called formal type parameters), separated by commas.
- A type parameter, also known as a type variable, is an identifier that specifies a generic type name.
- Can be used to declare the return type, parameter types and local variable types in a generic method, and act as placeholders for the types of the arguments passed to the generic method (actual type arguments).
- A generic method's body is declared like that of any other method.
- Type parameters can represent only reference types—not primitive types.

Generic Methods: Implementation and Compile-Time Translation

Printing array elements using generic method
`PrintArray`

```
// generic method printArray
public static <T> void printArray(T[] inputArray) {
    // display array elements
    for (T element : inputArray)
        System.out.printf("%s ", element);
    System.out.println();
} // end method printArray
```

Generic Methods: Implementation and Compile-Time Translation

```
public class GenericMethodTest {  
    public static void main(String[] args) {  
        // create arrays of Integer, Double and Character  
        Integer[] intArray = { 1, 2, 3, 4, 5 };  
        Double[] doubleArray = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7 };  
        Character[] charArray = { 'H', 'E', 'L', 'L', 'O' };  
  
        System.out.println("Array integerArray contains:");  
        printArray(intArray); // pass an Integer array  
  
        System.out.println("\nArray doubleArray contains:");  
        printArray(doubleArray); // pass a Double array  
  
        System.out.println("\nArray characterArray contains:");  
        printArray(charArray); // pass a Character array  
    } // end main  
}
```

Generic Methods: Implementation and Compile-Time Translation

- ☾ When the compiler translates generic method `printArray` into Java bytecodes, it removes the type-parameter section and replaces the type parameters with actual types.
- ☾ This process is known as erasure.
- ☾ By default all generic types are replaced with type `Object`.

Generic Methods: Implementation and Compile-Time Translation

Before compilation

```
public static <T> void printArray(T[] array) {  
    for (T element : array) {  
        System.out.println(element);  
    }  
}
```

After compilation

```
public static void printArray(Object[] array) {  
    for (Object element : array) {  
        System.out.println(element);  
    }  
}
```

To be continued ... next week

Generic methods and classes

Additional Compile-Time Translation Issues

- Generic method `maximum` determines and return the largest of its three arguments of the same type.
- The relational operator `>` cannot be used with reference types, but it's possible to compare two objects of the same class if that class implements the generic interface **`Comparable<T>`** (package `java.lang`).
- All the type-wrapper classes for primitive types implement this interface.
- Like generic classes, generic interfaces enable you to specify, with a single interface declaration, a set of related types.

Additional Compile-Time Translation Issues

- ☾ `Comparable<T>` objects have a `compareTo` method.

The method must return 0 if the objects are equal, a negative integer if `object1` is less than `object2` or a positive integer if `object1` is greater than `object2`.

- ☾ A benefit of implementing interface `Comparable<T>` is that `Comparable<T>` objects can be used with the sorting and searching methods of class `Collections` (package `java.util`).

Additional Compile-Time Translation Issues

```
public class MaximumTest
{
    public static void main( String[] args )
    {
        System.out.printf( "Maximum of %d, %d and %d is %d\n\n", 3, 4, 5,
            maximum( 3, 4, 5 ) );
        System.out.printf( "Maximum of %.1f, %.1f and %.1f is %.1f\n\n",
            6.6, 8.8, 7.7, maximum( 6.6, 8.8, 7.7 ) );
        System.out.printf( "Maximum of %s, %s and %s is %s\n", "pear",
            "apple", "orange", maximum( "pear", "apple", "orange" ) );
    } // end main
}
```

Additional Compile-Time Translation Issues

```
// determines the largest of three Comparable objects
public static <T extends Comparable<T>> T maximum( T x, T y, T z )
{
    T max = x; // assume x is initially the largest
    if ( y.compareTo( max ) > 0 )
        max = y; // y is the largest so far
    if ( z.compareTo( max ) > 0 )
        max = z; // z is the largest
    return max; // returns the largest object
} // end method maximum
```

Maximum of 3, 4 and 5 is 5

Maximum of 6.6, 8.8 and 7.7 is 8.8

Maximum of pear, apple and orange is pear

Additional Compile-Time Translation Issues

- ★ The type-parameter section specifies that **T extends Comparable<T>** - only objects of classes that implement interface Comparable<T> can be used with this method.
- ★ Comparable is known as the upper bound of the type parameter.
- ★ By default, Object is the upper bound.
- ★ Type-parameter declarations that bound the parameter always use keyword extends regardless of whether the type parameter extends a class or implements an interface.
- ★ The restriction of using **Comparable<T>** objects is important, because not all objects can be compared.

```
public class Student implements Comparable<Student>
{
    private String name;
    private double gpa;
    public Student(String name, double gpa)
    {
        this.name = name;
        this.gpa = gpa;
    }
    public String getName()
    {
        return name;
    }
    public double getGpa()
    {
        return gpa;
    }
    @Override
    public int compareTo(Student other)
    {
        if (this.gpa > other.gpa)
            return 1;
        else if (this.gpa < other.gpa)
            return -1;
        else
            return 0;
    }
}
```



You can also use it for your own class, such as Student, as long as you define **what "compare" means** for students!

```
public class TestStudentCompare
{
    public static void main(String[] args)
    {
        Student s1 = new Student("Anna", 3.6);
        Student s2 = new Student("Ben", 3.8);

        if (s1.compareTo(s2) < 0)
            System.out.println(s1.getName() + " has lower GPA than " + s2.getName());
        else if (s1.compareTo(s2) > 0)
            System.out.println(s1.getName() + " has higher GPA than " + s2.getName());
        else
            System.out.println("They have the same GPA");
    }
}
```

Additional Compile-Time Translation Issues

- ✧ When the compiler translates generic method maximum into Java bytecodes, it uses erasure to replace the type parameters with actual types.
- ✧ All type parameters are replaced with the upper bound of the type parameter, which is specified in the typeparameter section.
- ✧ When the compiler replaces the type-parameter information with the upper-bound type in the method declaration, it also inserts explicit cast operations in front of each method call to ensure that the returned value is of the type expected by the caller.

Additional Compile-Time Translation Issues

Generic method maximum after Erasure is performed by the compiler.

```
public static Comparable maximum(Comparable x, Comparable y, Comparable z)
{
    Comparable max = x; // assume x is initially the largest
    if (y.compareTo(max) > 0)
    {
        max = y; // y is the largest so far
    }
    if (z.compareTo(max) > 0)
    {
        max = z; // z is the largest
    }
    return max; // returns the largest object
}
```

Overloading Generic Methods

- ☾ A generic method may be overloaded.
- ☾ A class can provide two or more generic methods that specify the same method name but different method parameters.
- ☾ A generic method can also be overloaded by non-generic methods.
- ☾ When the compiler encounters a method call, it searches for the method declaration that most precisely matches the method name and the argument types specified in the call.

Overloading Generic Methods

```
public class Demo
{
    public static <T> void print(T value)
    {
        System.out.println("Generic method: " + value);
    }

    public static void print(String text)
    {
        System.out.println("Non-generic String method: " + text);
    }
}
```

Wildcard-type arguments

- ✧ Suppose we want to implement a generic method `sum` that calculates the total of numbers stored in an `ArrayList`.
- ✧ The collection may contain different numeric values.
- ✧ Primitive values such as `int` and `double` are autoboxed into wrapper objects like `Integer` and `Double`.
- ✧ We want the method to total all values regardless of their specific numeric type.
- ✧ Therefore, we may first think of using `ArrayList<Number>`, since `Number` is the superclass of `Integer` and `Double`.
- ✧ The method `sum` can then receive an `ArrayList<Number>` and compute the total of its elements.

Wildcard-type arguments

```
public class TotalNumbers
{
    public static void main(String[] args)
    {
        // create, initialize and output ArrayList of Numbers containing
        // both Integers and Doubles, then display total of the elements
        Number[] numbers = {1, 2.4, 3, 4.1}; // Integers and Doubles
        ArrayList<Number> numberList = new ArrayList<Number>();

        for (Number element : numbers)
            numberList.add(element); // place each number in numberList

        System.out.printf("numberList contains: %s\n", numberList);
        System.out.printf("Total of the elements in numberList: %.1f\n",
            sum(numberList));
    } // end main
}
```

In method `sum`: The `for` statement assigns each `Number` in the `ArrayList` to variable `element`, then uses `Number` method `doubleValue` to obtain the `Number`'s underlying primitive value as a double value. The result is added to `total`. When the loop terminates, the method returns the `total`.

```
// calculate total of ArrayList elements
public static double sum(ArrayList<Number> list)
{
    double total = 0; // initialize total

    // calculate sum
    for (Number element : list)
        total += element.doubleValue();

    return total;
} // end method sum
```

Wildcard-type arguments

- ✧ The method `sum(ArrayList<Number>)` works for a list of `Number` objects. But it does not accept `ArrayList<Integer>` or `ArrayList<Double>`.
- ✧ Even though `Integer` is a subclass of `Number`, `ArrayList<Integer>` is not a subclass of `ArrayList<Number>`.
- ✧ Java generics are invariant, so these parameterized types are treated as different types. If Java allowed this directly, type safety could be broken.
- ✧ Therefore, we use wildcards to make generic methods more flexible while preserving safety.

Wildcard-type arguments

Upper Bounded Wildcard (<? extends T>)

```
public static double sum(ArrayList<? extends Number> list)
{
    double total = 0;

    for (Number element : list)
        total += element.doubleValue();

    return total;
}
```

Wildcard-type arguments

To create a more flexible version of the `sum` method that can total the elements of any **ArrayList** containing elements of any subclass of **Number** we use wildcard-type arguments.

Wildcards enable you to specify method parameters, return values, variables or fields, and so on, that act as supertypes or subtypes of parameterized types.

- ✧ A wildcard-type argument is denoted by a question mark (?), which by itself represents an “unknown type.”
 - ▶ In this case, the wildcard extends class **Number**, which means that the wildcard has an upper bound of **Number**.
 - ▶ Thus, the unknown-type argument must be either **Number** or a subclass of **Number**.

Wildcard-type arguments

```
public class WildcardTest
{
    public static void main( String[] args )
    {
        // create, initialize and output ArrayList of Integers, then
        // display total of the elements
        Integer[] integers = { 1, 2, 3, 4, 5 };
        ArrayList<Integer> integerList = new ArrayList<Integer>();

        // insert elements in integerList
        for ( Integer element : integers )
            integerList.add( element );

        System.out.printf( "integerList contains: %s\n", integerList );
        System.out.printf( "Total of the elements in integerList: %.0f\n\n",
            sum( integerList ) );
    }
}
```

Wildcard-type arguments

```
// create, initialize and output ArrayList of Doubles, then
// display total of the elements
Double[] doubles = { 1.1, 3.3, 5.5 };
ArrayList<Double> doubleList = new ArrayList<Double>();

// insert elements in doubleList
for ( Double element : doubles )
    doubleList.add( element );

System.out.printf( "doubleList contains: %s\n", doubleList );
System.out.printf( "Total of the elements in doubleList: %.1f\n\n",
    sum( doubleList ) );
```

Wildcard-type arguments

```
// create, initialize and output ArrayList of Numbers containing
// both Integers and Doubles, then display total of the elements
Number[] numbers = { 1, 2.4, 3, 4.1, 5 };
ArrayList<Number> numberList = new ArrayList<Number>();
// insert elements in numberList
for ( Number element : numbers )
    numberList.add( element );

System.out.printf( "numberList contains: %s\n", numberList );
System.out.printf( "Total of the elements in numberList: %.1f\n",
    sum( numberList ) );
} // end main
```

Wildcard-type arguments

```
// total the elements; using a wildcard in the ArrayList parameter
public static double sum( ArrayList<? extends Number> list )
{
    double total = 0; // initialize total

    // calculate sum
    for ( Number element : list )
        total += element.doubleValue();

    return total;
} // end method sum
} // end class WildcardTest
```

Lower Bounded Wildcard (<? super T>)

```
public class LowerBoundedWildcardDemo
{
    public static void main(String[] args)
    {
        ArrayList<Number> numberList = new ArrayList<Number>();
        ArrayList<Object> objectList = new ArrayList<Object>();

        addIntegers(numberList);
        addIntegers(objectList);

        System.out.println("numberList: " + numberList);
        System.out.println("objectList: " + objectList);
    }

    // lower bounded wildcard: accepts Integer or any superclass of Integer
    public static void addIntegers(ArrayList<? super Integer> list)
    {
        list.add(10);
        list.add(20);
        list.add(30);
    }
}
```

Generic Class

- ☾ We want a class that:
- ▶ Works with **any data type**
 - ▶ Ensures **type safety**
 - ▶ Avoids **casting**
 - ▶ Detects errors at **compile-time**

Generic Class

- ★ A generic class is a class that is defined with one or more type parameters. Instead of fixing the data type when the class is written, the type is supplied later when an object is created. This allows the class to be reused for many different data types while preserving type safety.
- ★ A generic class is useful when the structure and behavior of a class remain the same, but the type of data stored or processed may vary.

Without Generic Class

Declare

```
class Box {  
    private Object data;  
  
    public void setData(Object data) {  
        this.data = data;  
    }  
    public Object getData() {  
        return data;  
    }  
}
```

Use

```
Box box = new Box();  
box.setData("Hello");  
String s = (String) box.getData();
```

```
Box box = new Box();  
box.setData(100);  
String s = (String) box.getData();
```



Exception in thread "main"
java.lang.ClassCastException:
class java.lang.Integer cannot
be cast to class java.lang.String

Generic Classes - Structure

A generic class is a class that operates on a type specified at instantiation time.

```
class Box<T> {  
    private T value;  
  
    public void set(T value) {  
        this.value = value;  
    }  
  
    public T get() {  
        return value;  
    }  
}
```

Type instantiation

```
Box<String> box1 = new Box<>();  
box1.set("Hello");
```

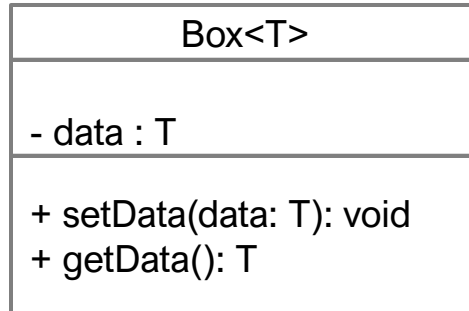
→ replaces T with String

```
Box<Integer> box2 = new Box<>();  
box2.set(100);
```

→ replaces T with Integer

UML for Generic Class

- ☾ <T> is shown at the class level
- ☾ All occurrences of T are consistent within the class
- ☾ UML explicitly shows parametric abstraction



Multiple Type Parameters

By default:

- T = Type
- E = Element
- K = Key
- V = Value

```
class Pair<K, V> {  
    private K key;  
    private V value;  
  
    public Pair(K key, V value) {  
        this.key = key;  
        this.value = value;  
    }  
  
    public K getKey() {  
        return key;  
    }  
  
    public V getValue() {  
        return value;  
    }  
}
```

```
Pair<String, Integer> p = new Pair<>("Age", 20);
```

```
System.out.println(p.getKey()); // Age  
System.out.println(p.getValue()); // 20
```